



TECNOLÓGICO
NACIONAL DE MÉXICO



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



INSTITUTO TECNOLÓGICO
DE CIUDAD MADERO

ITCM

INGENIERIA EN SISTEMAS COMPUTACIONALES
SIMULACIÓN

PRÁCTICA NO.4

**“GENERACIÓN DE NÚMEROS
ALEATORIOS”**

PT.2 DE 2

UNIDAD 1 | E1A | 12:00 a 3:00

ALUMNO

- Izaguirre Rodríguez Ana María

30 de Junio del 2026

Calificación: _____

Contenido

1	INSTRUCCIONES	3
2	NOTAS	3
2.1	PASO 1	3
2.1.1	Definición de Parámetros Clave.....	3
2.2	PASO 2	3
2.2.1	Ejecución del Algoritmo Congruencial Mixto	3
2.3	PASO 3	3
2.3.1	Validación Estadística de Periodo Completo	3
2.4	PASO 4	3
2.4.1	Muestra de Control en Consola	3
2.5	PASO 5	3
2.5.1	Escritura Automatizada del Archivo de Reporte	3
3	DESARROLLO	4
3.1	CÓDIGO	4
3.1.1	CÓDIGO CON NOTAS (CAPTURAS)	4
3.1.2	CÓDIGO SIN NOTAS (TEXTO).....	6
3.2	EJECUCIÓN.....	9
3.2.1	RUN	9
3.2.2	ARCHIVO	10
4	CONCLUSIONES	14

1 INSTRUCCIONES

Utilice uno de los métodos de generación vistos y construya un programa en un lenguaje de alto nivel en que, dados los datos que requiera, genere un período completo de al menos 4096 números pseudoaleatorios. Guarde en un archivo la semilla, las constantes y el módulo utilizado. Entregar reporte.

2 NOTAS

2.1 PASO 1

2.1.1 Definición de Parámetros Clave

Se configuran las variables esenciales del modelo congruencial mixto. El módulo se establece en $M = 4096$ para garantizar el tamaño mínimo solicitado por la práctica. Los valores del multiplicador ($a = 17$) y la constante aditiva ($b = 9$) fueron seleccionados bajo criterios matemáticos estrictos: b es primo relativo con M , y $(a-1)$ es múltiplo de 4, lo cual asegura teóricamente que el algoritmo alcance su periodo máximo antes de sufrir repeticiones.

2.2 PASO 2

2.2.1 Ejecución del Algoritmo Congruencial Mixto

Se utiliza un ciclo controlado de tipo for que itera exactamente 4096 veces. En cada ciclo, se aplica la ecuación fundamental Lehmer: $X_i = (a * X_{i-1} + b) \pmod{M}$. Este residuo matemático se almacena en la estructura dinámica numeroGenerados, y el valor resultante se vuelve la semilla de entrada de la iteración inmediata posterior.

2.3 PASO 3

2.3.1 Validación Estadística de Periodo Completo

Para garantizar que no ocurra un ciclo o desgaste anticipado en la serie pseudoaleatoria, el sistema implementa un filtro de auditoría mediante una lista auxiliar llamada verificador. El programa recorre todos los valores y evalúa si alguno ya existía en memoria; al confirmar de manera automática que no hay duplicados y que el total de números únicos es idéntico al módulo ($M = 4096$), se valida con éxito el Periodo Completo.

2.4 PASO 4

2.4.1 Muestra de Control en Consola

Para una inspección visual veloz en la terminal del entorno de desarrollo, el código imprime de forma formateada los primeros 15 elementos generados. En este paso se realiza además la transformación matemática estándar de los residuos dividiéndolos entre el módulo ($r_i = \frac{X_i}{M}$) convirtiendo los números enteros en fracciones uniformes distribuidas estrictamente entre el 0 y la unidad (1), tal como lo requiere la simulación estocástica aplicada.

2.5 PASO 5

2.5.1 Escritura Automatizada del Archivo de Reporte

Mediante los objetos nativos de flujo de salida de Java (FileWriter y PrintWriter), el programa genera de manera física un archivo externo de extensión .txt en la raíz del proyecto. El script vuelca las constantes del sistema y tabula de forma organizada los 4096 registros con sus respectivos números fraccionarios uniformes, proporcionando un archivo persistente y estructurado.

3 DESARROLLO

3.1 CÓDIGO

3.1.1 CÓDIGO CON NOTAS (CAPTURAS)

```

package SIMULACION;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.List;
public class SU1P4E1 {
public static void main(String[] args) {

    int semilla = 7326;
    int a = 17;
    int b = 9;
    int M = 4096;

    List<Integer> numeroGenerados = new ArrayList<>();
    int actual = semilla;

    System.out.println("=== INICIANDO GENERACIÓN DE " + M + " NÚMEROS PSEUDOALEATORIOS ===");

    for (int i = 0; i < M; i++) {
        int siguiente = (a * actual + b) % M;
        numeroGenerados.add(siguiente);
        actual = siguiente;
    }

    boolean tieneDuplicados = false;
    List<Integer> verificador = new ArrayList<>();
    for (int num : numeroGenerados) {
        if (verificador.contains(num)) {
            tieneDuplicados = true;
            break;
        }
        verificador.add(num);
    }

    System.out.println("\n--> VALIDACIÓN ESTADÍSTICA DE LA CORRIDA:");
    System.out.println("    ¿Tiene números repetidos antes de terminar el periodo?: " + (tieneDuplicados ? "Sí (Ajustar constantes)" : "NO (Periodo"));
    System.out.println("    Total de números únicos en la serie: " + verificador.size());
    System.out.println("    : "NO (Periodo Completo Exitoso)");

    System.out.println("\nMUESTRA INICIAL DE LA SERIE:");
    for (int i = 0; i < 15; i++) {
        System.out.printf("X_ %d = %d (Fracción: %.4f)\n", (i + 1), numeroGenerados.get(i), (double)numeroGenerados.get(i)/M);
    }

    String nombreArchivo = "RSU1P4E1.txt";
    try (FileWriter fw = new FileWriter(nombreArchivo);
        PrintWriter pw = new PrintWriter(fw)) {

        pw.println("=====");
        pw.println("    REPORTE DE GENERADOR DE NÚMEROS PSEUDOALEATORIOS");
        pw.println("=====");
        pw.println("PARAMETROS UTILIZADOS:");
        pw.println("-> Semilla Inicial (X0): " + semilla);
        pw.println("-> Constante Multiplicativa (a): " + a);
        pw.println("-> Constante Aditiva (b): " + b);
        pw.println("-> Modulo Seleccionado (M): " + M);
        pw.println("-> Tipo de Periodo: Completo (4096 valores unicos)");
        pw.println("=====");
        pw.println(String.format("| %-6s | %-12s | %-15s |", "i", "Valor (X_i)", "Num. Aleatorio (r_i)"));
        pw.println("-----");

        for (int i = 0; i < numeroGenerados.size(); i++) {
            int valor = numeroGenerados.get(i);
            double r_i = (double) valor / M;
            pw.println(String.format("| %-6d | %-12d | %-20.6f |", (i + 1), valor, r_i));
        }

        pw.println("-----");
        pw.println("FIN DEL REPORTE - GENERACIÓN EXITOSA.");
    }
}

```

Definición de parámetros para cumplir con el periodo completo de 4096

Ejecución del algoritmo congruencial mixto

Validación de periodo completo en consola (Control de calidad)

Muestra de los primeros 15 números en consola para inspección visual

Escritura física del archivo de texto requerido

Cabecera con las constantes solicitadas en las instrucciones

Volcado de toda la lista de los 4096 números al archivo txt

3.1.2 CÓDIGO SIN NOTAS (TEXTO)

```
package SIMULACION;

import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.List;

public class SU1P4E1 {

    public static void main(String[] args) {

        int semilla = 7326;

        int a = 17;

        int b = 9;

        int M = 4096;

        List<Integer> numeroGenerados = new ArrayList<>();

        int actual = semilla;

        System.out.println("=== INICIANDO GENERACIÓN DE " + M + " NÚMEROS
PSEUDOALEATORIOS ===");

        for (int i = 0; i < M; i++) {

            int siguiente = (a * actual + b) % M;

            numeroGenerados.add(siguiente);

            actual = siguiente;

        }

    }

}
```

```

boolean tieneDuplicados = false;

List<Integer> verificador = new ArrayList<>();

for (int num : numeroGenerados) {
    if (verificador.contains(num)) {
        tieneDuplicados = true;
        break;
    }
    verificador.add(num);
}

System.out.println("\n--> VALIDACIÓN ESTADÍSTICA DE LA CORRIDA:");

System.out.println("    ¿Tiene números repetidos antes de terminar el periodo?: " +
(tieneDuplicados ? "SÍ (Ajustar constantes)" : "NO (Periodo Completo Exitoso)"));

System.out.println("    Total de números únicos en la serie: " + verificador.size());


System.out.println("\nMUESTRA INICIAL DE LA SERIE:");

for (int i = 0; i < 15; i++) {
    System.out.printf("X_%d = %d (Fracción: %.4f)%n", (i + 1), numeroGenerados.get(i),
(double)numeroGenerados.get(i)/M);
}


String nombreArchivo = "RSU1P4E1.txt";

try (FileWriter fw = new FileWriter(nombreArchivo);

    PrintWriter pw = new PrintWriter(fw)) {

    pw.println("=====");

    pw.println("    REPORTE DE GENERADOR DE NÚMEROS PSEUDOALEATORIOS");

```

```

        pw.println("=====");

        pw.println("PARAMETROS UTILIZADOS:");

        pw.println("-> Semilla Inicial (X0): " + semilla);

        pw.println("-> Constante Multiplicativa (a): " + a);

        pw.println("-> Constante Aditiva (b): " + b);

        pw.println("-> Modulo Seleccionado (M): " + M);

        pw.println("-> Tipo de Periodo: Completo (4096 valores unicos)");

        pw.println("=====");

        pw.println(String.format("| %-6s | %-12s | %-15s |", "i", "Valor (X_i)", "Num. Aleatorio (r_i)"));

        pw.println("-----");

        for (int i = 0; i < numeroGenerados.size(); i++) {

            int valor = numeroGenerados.get(i);

            double r_i = (double) valor / M;

            pw.println(String.format("| %-6d | %-12d | %-20.6f |", (i + 1), valor, r_i));

        }

        pw.println("-----");

        pw.println("FIN DEL REPORTE - GENERACIÓN EXITOSA.");

        System.out.println("\n[ÉXITO] Archivo " + nombreArchivo + " generado correctamente en la carpeta raíz del proyecto.");

    } catch (IOException e) {

        System.out.println("\n[ERROR] No se pudo escribir el archivo de reporte: " + e.getMessage());

    }

}

}

```


3.2 EJECUCIÓN

3.2.1 RUN

```
--- exec:3.1.0:exec (default-cli) @ marsito ---
=== INICIANDO GENERACIÓN DE 4096 NÚMEROS PSEUDOALEATORIOS ===

--> VALIDACIÓN ESTADÍSTICA DE LA CORRIDA:
    Tiene números repetidos antes de terminar el periodo?: NO (Periodo Completo Exitoso)
    Total de números únicos en la serie: 4096

MUESTRA INICIAL DE LA SERIE:
X_1 = 1671 (Fracción: 0.4080)
X_2 = 3840 (Fracción: 0.9375)
X_3 = 3849 (Fracción: 0.9397)
X_4 = 4002 (Fracción: 0.9771)
X_5 = 2507 (Fracción: 0.6121)
X_6 = 1668 (Fracción: 0.4072)
X_7 = 3789 (Fracción: 0.9250)
X_8 = 2982 (Fracción: 0.7280)
X_9 = 1551 (Fracción: 0.3787)
X_10 = 1800 (Fracción: 0.4395)
X_11 = 1937 (Fracción: 0.4729)
X_12 = 170 (Fracción: 0.0415)
X_13 = 2899 (Fracción: 0.7078)
X_14 = 140 (Fracción: 0.0342)
X_15 = 2389 (Fracción: 0.5833)

[ÉXITO] Archivo 'RSU1P4E1.txt' generado correctamente en la carpeta raíz del proyecto.
-----
BUILD SUCCESS
-----
Total time: 2.305 s
Finished at: 2026-06-29T14:29:09-06:00
-----
```


TRABAJO			
CUsersgoc			
RSU1P4E1			
Archivo	Editar	Ver	H1
27	1185		0.289307
28	3770		0.920410
29	2659		0.649170
30	156		0.038086
31	2661		0.649658
32	190		0.046387
33	3239		0.790771
34	1824		0.445313
35	2345		0.572510
36	3010		0.734863
37	2027		0.494873
38	1700		0.415039
39	237		0.057861
40	4038		0.985840
41	3119		0.761475
42	3880		0.947266
43	433		0.105713
44	3274		0.799316
45	2419		0.590576
46	172		0.041992
47	2933		0.716064
48	718		0.175293
49	4023		0.982178
50	2864		0.699219
51	3641		0.888916
52	466		0.113770
53	3835		0.936279
54	3764		0.918945
55	2557		0.624268
56	2518		0.614746
57	1855		0.452881
58	2872		0.701172
59	3777		0.922119
60	2778		0.678223
61	2179		0.531982
62	188		0.045898
63	3205		0.782471
64	1246		0.304199
65	711		0.173584
66	3904		0.953125

Ln 1, Col 1

201,339 caracte

Texto sin form

100%

Windows (CRLF

UTF-8

3.2.2.2 Últimas Dos

TRABAJOS • CUsersgod • RSU1P4E1. X +			
Archivo	Editar	Ver	H1 ▾ ≡ ▾ B ...
4018	2736	0.667969	
4019	1465	0.357666	
4020	338	0.082520	
4021	1659	0.405029	
4022	3636	0.887695	
4023	381	0.093018	
4024	2390	0.583496	
4025	3775	0.921631	
4026	2744	0.669922	
4027	1601	0.390869	
4028	2650	0.646973	
4029	3	0.000732	
4030	60	0.014648	
4031	1029	0.251221	
4032	1118	0.272949	
4033	2631	0.642334	
4034	3776	0.921875	
4035	2761	0.674072	
4036	1890	0.461426	
4037	3467	0.846436	
4038	1604	0.391602	
4039	2701	0.659424	
4040	870	0.212402	
4041	2511	0.613037	
4042	1736	0.423828	
4043	849	0.207275	
4044	2154	0.525879	
4045	3859	0.942139	
4046	76	0.018555	
4047	1301	0.317627	
4048	1646	0.401855	
4049	3415	0.833740	
4050	720	0.175781	
4051	4057	0.990479	
4052	3442	0.840332	
4053	1179	0.287842	
4054	3668	0.895508	
4055	925	0.225830	
4056	3446	0.841309	
4057	1247	0.304443	
Ln 1, Col 1 201,339 caracte Texto sin form 100% Windows (CRLF UTF-8			

TRABAJOS • CUsersgod • RSU1P4E1. X			
Archivo	Editar	Ver	H1
4058	728		0.177734
4059	97		0.023682
4060	1658		0.404785
4061	3619		0.883545
4062	92		0.022461
4063	1573		0.384033
4064	2174		0.530762
4065	103		0.025146
4066	1760		0.429688
4067	1257		0.306885
4068	898		0.219238
4069	2987		0.729248
4070	1636		0.399414
4071	3245		0.792236
4072	1926		0.470215
4073	4079		0.995850
4074	3816		0.931641
4075	3441		0.840088
4076	1162		0.283691
4077	3379		0.824951
4078	108		0.026367
4079	1845		0.450439
4080	2702		0.659668
4081	887		0.216553
4082	2800		0.683594
4083	2553		0.623291
4084	2450		0.598145
4085	699		0.170654
4086	3700		0.903320
4087	1469		0.358643
4088	406		0.099121
4089	2815		0.687256
4090	2808		0.685547
4091	2689		0.656494
4092	666		0.162598
4093	3139		0.766357
4094	124		0.030273
4095	2117		0.516846
4096	3230		0.788574

4 CONCLUSIONES

Al finalizar el desarrollo de esta práctica, se diseñó e implementó exitosamente un generador de números pseudoaleatorios utilizando el Enfoque Congruencial Lineal Mixto en un lenguaje de alto nivel. El principal hallazgo técnico radica en la comprobación empírica del Teorema de Periodo Completo de Lehmer. Mediante la selección matemática rigurosa de las constantes, donde la constante aditiva b es un número primo relativo respecto al módulo $M = 4096$, y el multiplicador a cumple de forma idónea las directrices de potencia entera de dos, se logró que el programa generara la totalidad del universo posible de valores únicos en una secuencia estadísticamente homogénea antes de repetir cualquier dato de su historial. Asimismo, el programa demostró la efectividad de la transformación matemática para normalizar números enteros en variables uniformes representadas como fracciones decimales estrictas entre 0 y 1. Este tipo de automatización de datos persistentes a través de archivos de texto (.txt) es una competencia técnica crítica para la ingeniería de sistemas, pues reduce de forma drástica el consumo de memoria en hardware y provee un suministro masivo, rápido y confiable de números aleatorios idóneos para alimentar futuras fases experimentales en modelos probabilísticos y metodologías de Montecarlo.